Videojuegos

GRADO EN INGENIERÍA MULTIMEDIA

Sesión 03.1
Programación avanzada

2026-27

Programación avanzada

1. Traducir+emitir bytes
2. SECTION, DS, WRAM0, VRAM
3. Constantes con DEF
4. Comparar direcciones `HIGH()/LOW()`
5. Centinelas

Programación avanzada

1. Traducir+emitir bytes
2. SECTION, DS, WRAM0, VRAM
3. Constantes con DEF
4. Comparar direcciones `HIGH()/LOW()`
5. Centinelas
6. Devolver flags
7. Menor-que (C/NC)
8. Clobbering
9. Sumar negativos
10. Suma/Resta en 8 bits

1. Emitir bytes

Contador de traducción (@): traduce instrucciones, emite bytes y avanza.

- Una `etiqueta` = valor del contador.

```
SECTION "Code", ROM0[$0150]

main::                  ;; main = $ = $0150
   ld a, 5              ;; emite 3E 05     (2 bytes)
   ld [$C000], a        ;; emite EA 00 C0  (3 bytes)
   jr main              ;; emite 18 F9     (2 bytes)
code_end::              ;; code_end = $0157
   DB $C9               ;; emite C9 ('ret' 1 byte)
afterwards::            ;; afterwards = $0158

DEF CODE_SIZE = @ - main          ;; = 8 bytes
DEF LOOP_SIZE = code_end - main   ;; = 7 bytes
```

2. SECTION, DS, WRAM0 y VRAM

No podemos emitir bytes a RAM

Los valores iniciales son aleatorios.

```
SECTION "Game Data", WRAM0
player_x: DS 1          ;; player_x = $C000
player_y: DS 1          ;; player_y = $C001
enemies:  DS 8          ;; enemies  = $C002
score:    DS 2          ;; score    = $C00A

SECTION "Sprites", ROM0
sprite_data: DB $3C, $42, $81, $81   ;; ROM: SÍ lleva datos

SECTION "Code", ROM0
init:
   xor a
   ld [player_x], a     ;; RAM: valor inicial ¡por código!
```

3. Constantes con DEF

- Son ***símbolos***
- Sólo existen ***durante la traducción***
- Si son globales, llegan al ***linker***.

```
DEF BOARD_WIDTH  = 8
DEF BOARD_HEIGHT = 12
DEF BOARD_SIZE   = BOARD_WIDTH * BOARD_HEIGHT ;; 96
EXPORT DEF TILEMAP EQU $9800                  ;; Global (linker)

main::                                  ;; main global (linker)
   ld  b, BOARD_HEIGHT                  ;; == ld b, 12
   ld hl, TILEMAP + 3*32 + 5            ;; fila 3, columna 5
end_code:                               ;; end_code local (no linker)
```

4. Comparar direcciones

```
SECTION "Board", WRAM0[$C100]
  board:      DS $96    ;; board = $C100
  board_end:            ;; board_end = $C196
SECTION "Code", ROM0    ;;  ...justo DESPUÉS de la última celda
clear_board:
   ld  hl, board        ;; hl = $C100
.loop:
   xor a                ;; A = 0
   ld  [hl+], a
   ld  a, l              ;; Comparar HL == board_end??
   cp  LOW(board_end)    ;; ¿parte baja igual?
   jr  nz, .loop
   ld  a, h
   cp  HIGH(board_end)  ;; ¿parte alta igual?
   jr  nz, .loop         ;; hl == board_end: fin
   ret
```

- Bucles sin contador
- Comparar con dirección de fin, ***byte a byte***.

5. Centinelas

Un **dato** en memoria que ***indica el final***

```
DEF SENTINEL = $FF  ;; $FF nunca se usará como dato válido
                    ;; - así se distingue que es un marcador
SECTION "Data", ROM0
enemy_list: DB 10, 25, 40, 77, SENTINEL

SECTION "Code", ROM0
process_enemies::
   ld  hl, enemy_list   ;; HL = dirección inicio lista enemigos
.loop:
   ld  a, [hl+]     ;; Leo un enemigo
   cp  SENTINEL     ;; ¿Es $FF (el centinela)?
   ret z            ;;  - Sí? Entonces no es un enemigo
                    ;;    sino el final de la lista
   .process_enemy
   ;; a != SENTINEL, significa que es un enemigo válido
   ;; lo procesamos aquí
   jr  .loop
```

6. Devolver flags

- Podemos devolver Z/NZ o C/NC
- Quien llama lo puede usar directamente.

```
at_goal:  ;; Z = el jugador está en la meta | NZ = no
   ld a, [player_x]
   ld b, a
   ld a, [goal_x]
   cp b   ;; Z si son iguales: ¡ya está en el flag!
   ret

main::
   call at_goal ;; ¿Está en el objetivo?
   jr z, .goal  ;; Si Z activo, está en el objetivo
   .not_goal

;; Como `settear` el flag a propósito (si hace falta)
;; set_nz:        set_z:
;;     or 1         xor a
;;     ret          ret
```

7. Repaso: menor-que

- **Restar *A - nn***
  - `cp  nn` (mo modifica A)
  - `sub  nn` (modifica A)

7. Repaso: menor-que

- **Restar *A - nn***
  - `cp  nn` (mo modifica A)
  - `sub  nn` (modifica A)

- Flag C indica ***acarreo*** ("llevarse 1")
  - Porque ***nn > A***
  - Restar algo mayor, produce ***negativo***

7. Repaso: menor-que

- **Restar *A - nn***
  - `cp  nn` (mo modifica A)
  - `sub  nn` (modifica A)

- Flag C indica ***acarreo*** ("llevarse 1")
  - Porque ***nn > A***
  - Restar algo mayor, produce ***negativo***

```
ld  a, [player_x]
cp  20
jr  c, .menor     ;; C:  20  > A  ( A <  20 )
.not_carry        ;; NC: 20 <= A  ( A >= 20 )
                  ;; Z:  20 == A
```

8. *Clobbering*

- Una rutina ***puede*** ***sobreescribir registros***.
- Documenta cuáles, y guarda lo que necesites.

```
;; ❌ Modifica: A, B, HL
clear_row:
   xor a
   ld  b, 8
.loop:
   ld  [hl+], a
   dec b
   jr  nz, .loop
   ret
main::
   ld   b, 5            ;; B = fila que aún necesito
   push bc              ;; la guardo...
   call clear_row       ;; ...esta rutina destruye B
   pop  bc              ;; ...y la recupero: B = 5
```

9. Restar sumando negativo

- No existe `sub hl, rr`.
- Para constantes, suma su ***negativo***.

```
DEF BOARD_WIDTH = 8

   ;; 16 bits: hl = hl - 8  (subir una fila)
   ld  bc, -BOARD_WIDTH    ;; -8 = $FFF8
   add hl, bc

   ;; 8 bits: a = a - 3
   add a, -3               ;; igual que sub 3
```

10. Suma/Resta en 8 bits

- Los 8 bits ***dan la vuelta*** (módulo 256)
- El flag C es el ***acarreo*** (lo que llevamos!).
- Suma de 16 bits: add + `adc`.

```
ld  a, 250
add a, 10          ;; A = 4    (260 - 256), C = 1
ld  a, 3
sub 5              ;; A = 254  ($FE),       C = 1

;; HL = HL + 300  (byte a byte)
ld  a, l
add a, LOW(300)  ;; parte baja: puede dar acarreo
ld  l, a
ld  a, h
adc a, HIGH(300) ;; parte alta + el acarreo
ld  h, a
```

Bonus

Alineamiento

Si la parte alta no varía, mira sólo la parte baja

```
;SECTION "Board", WRAM0[$C000$]   ;; Dirección a mano
SECTION "Board", WRAM0, ALIGN[8]  ;; La hace el linker en $xx00
board:      DS 96                 ;; < 256: no cruza el bloque
board_end:                        ;; H no cambia nunca

clear_board:
   ld  hl, board
.loop:
   xor a
   ld  [hl+], a
   ld  a, l
   cp  LOW(board_end)   ;; basta con la parte baja
   jr  nz, .loop
   ret
```

Videojuegos

GRADO EN INGENIERÍA MULTIMEDIA

Sesión 03.1
Programación avanzada

2026-27